

# 获取MPU6050数据（I2C）

---

## 获取MPU6050数据（I2C）

### 一、学习目标

#### I2C介绍

#### I2C基本参数

### 二、硬件搭建

### 三、实验步骤

#### 1. 打开SYSCONFIG配置工具

#### 2. 引脚参数配置

#### 3. 串口配置

#### 4. I2C协议使用

#### 5. 写入程序

#### 6. 编译

### 四、程序分析

### 五、实验现象

## 一、学习目标

---

1.学习了解iic通讯基本知识。

2.获取MPU6050数据。

## I2C介绍

IIC总线是一个双向的两线连续总线，它为集成电路之间提供通信线路。其意思是完成集成电路或功能单元之间信息交换的协议。

IIC模块接收和发送数据，并将数据从串行转换成并行，或并行转换成串行。可以开启或禁止中断。接口通过数据引脚(SDA)和时钟引脚(SCL)连接到IIC总线。允许连接到标准(高达100kHz)或快速(高达400kHz)的IIC总线。(数据线SDA和时钟SCL构成串行总线，可发送和接收数据)。

IIC总线在传输数据过程中共有三种类型信号，分别是：开始信号(START)、停止(结束)信号(STOP)、应答信号(ACK)。其次在没有进行数据传输时处于空闲状态。

## I2C基本参数

**速率：** I2C总线有标准模式（100 kbit/s）和快速模式（400 kbit/s）两种传输模式，还有更快的扩展模式和高速模式可供选择。

**器件地址：** 每个设备都有唯一的7位或10位地址，可以通过地址选择来确定与谁进行通信。

**总线状态：** I2C总线有五种状态，分别是空闲状态、起始信号、结束信号、响应信号、数据传输。

**数据格式：** I2C总线有两种数据格式，标准格式和快速格式。标准格式是8位数据字节加上1位ack/nack（应答/非应答）位，快速格式允许两个字节同时传输。

由于SCL和SDA线是双向的，它们也可能会由于外部原因（比如线路中的电容等）出现电平误差，而从而导致通信出错。因此，在IIC总线中，通常使用上拉电阻来保证信号线在空闲状态下的电平为高电平。

二、硬件搭建

MSPM0G系列的I2C支持主从模式，有7位地址位可以设置，支持 100kbps、400kbps、1Mbps 的 I2C 标准传输速率，并支持 SMBUS。无论是主机或者从机，发送和接收都有独立的8个字节FIFO。MSPM0 I2C 具有 8 字节 FIFO，对于控制器和目标模式会生成独立的中断，并支持 DMA。

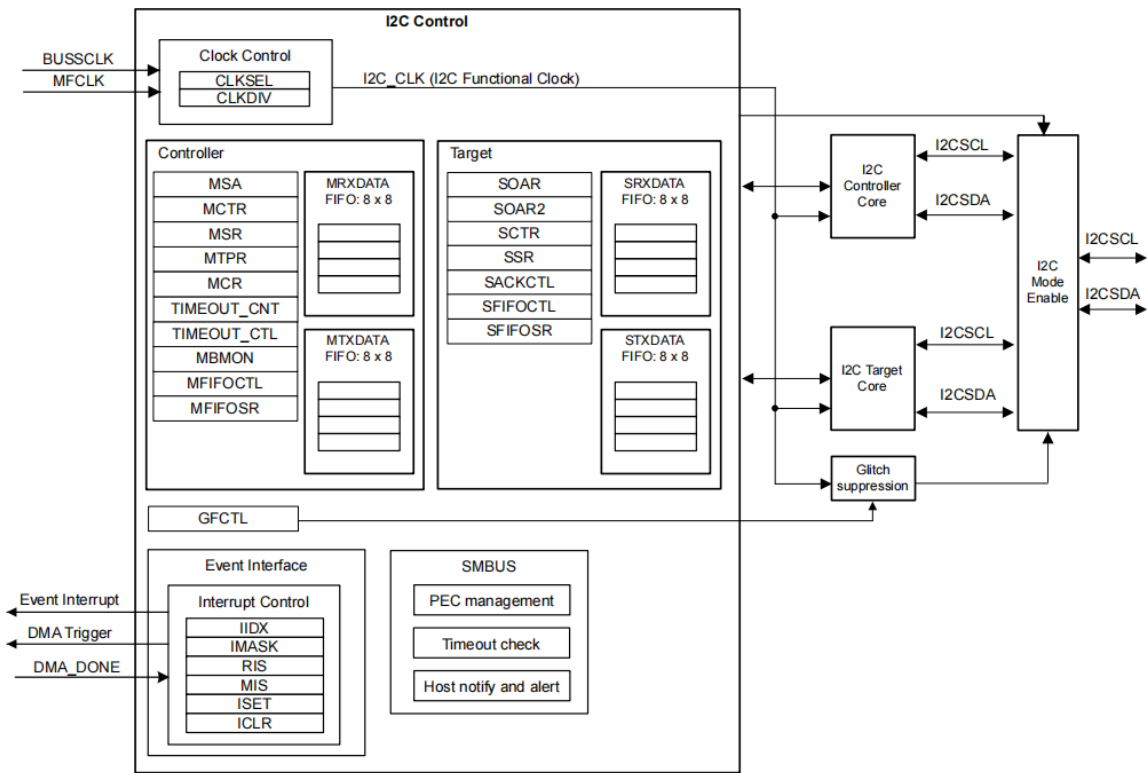


Figure 18-1. I2C Functional Block Diagram

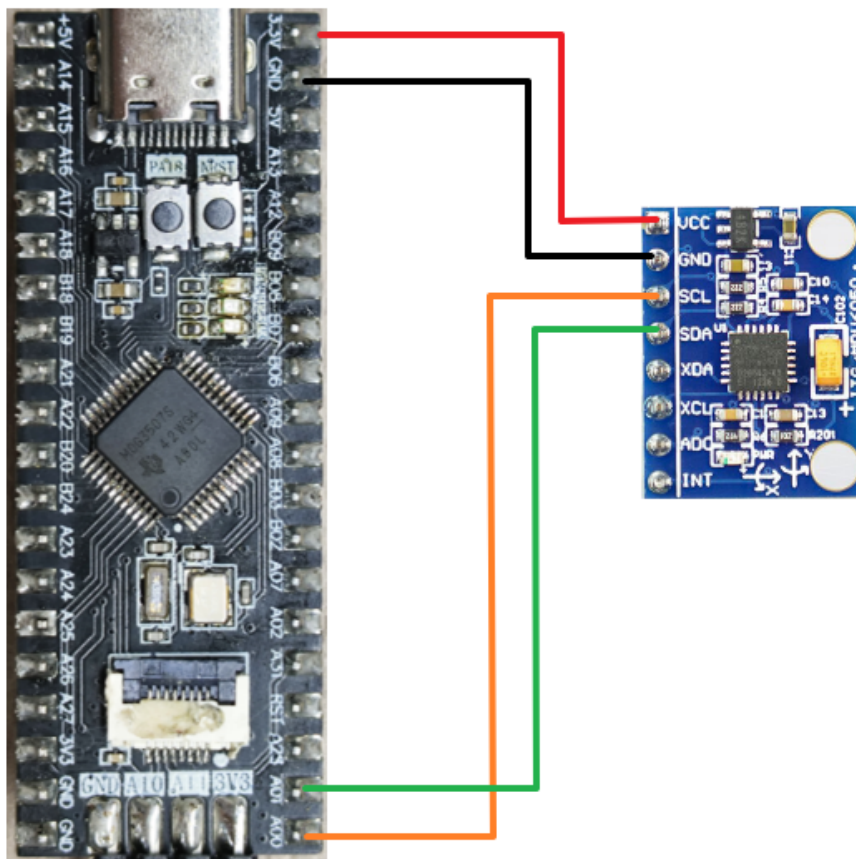
软件I2C是指通过在程序中编写代码来实现I2C通信协议。它利用通用输入输出（GPIO）引脚来模拟I2C的数据线（SDA）和时钟线（SCL），通过软件控制引脚的电平变化来传输数据和生成时序信号。与硬件I2C相比，软件I2C的优势在于不需要特定的硬件支持，可以在任何支持GPIO功能的微控制器上实现。它利用了微控制器的通用IO引脚来实现I2C通信协议。

硬件I2C是指通过专门的硬件模块来处理I2C通信协议。大多数现代微控制器和一些外部设备已经集成了硬件I2C模块，这些硬件模块负责处理I2C通信的细节，包括生成正确的时序信号、自动处理信号冲突、数据传输和错误检测等。可以直接使用硬件引脚连接，无需编写时序的代码。

本实验使用软件IIC来读取MPU6050模块的数据。

硬件连接

| MSPM0G3507 | MPU6050 |
|------------|---------|
| PA0        | SCL     |
| PA1        | SDA     |
| 3V3        | GND     |
| GND        | VCC     |



### 三、实验步骤

本课程将PA0、PA1引脚配置为SCL、SDA，读取MPU6050六轴传感器模块的数据。

这里使用我们提供的模板工程进行介绍。

将模板工程放在SDK的根目录下解压，我将其重新命名为10\_I2C文件夹。

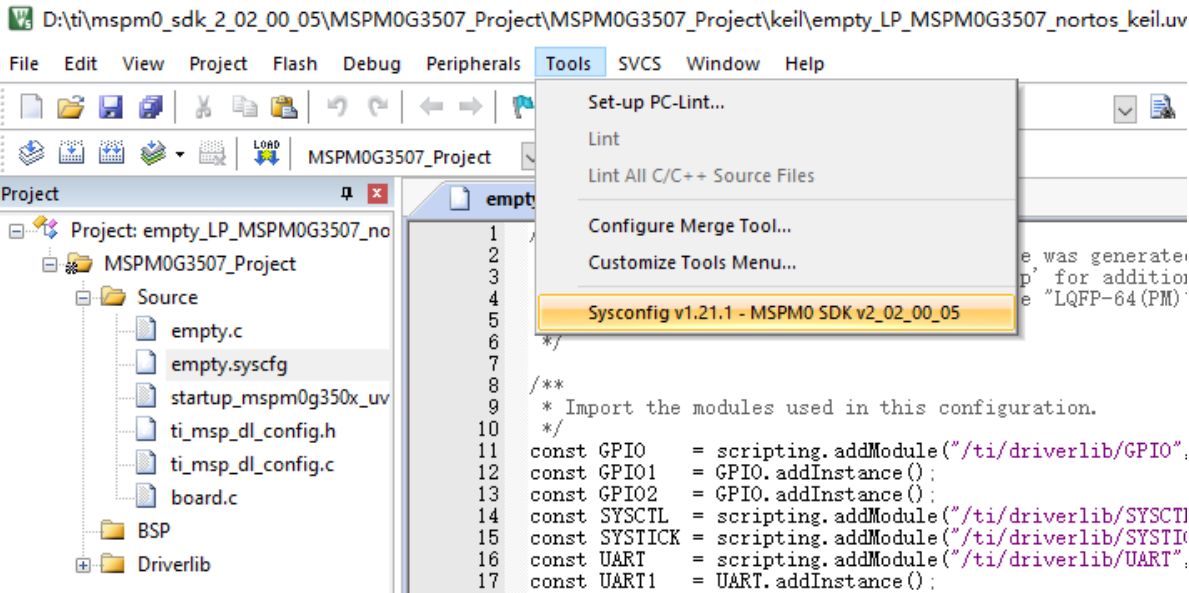
| :) > ti > mspm0_sdk_2_02_00_05 |                  |     |    |  | 在 mspm... |
|--------------------------------|------------------|-----|----|--|-----------|
| 名称                             | 修改日期             | 类型  | 大小 |  |           |
| 01_LED                         | 2024/11/19 11:42 | 文件夹 |    |  |           |
| 02_Delay                       | 2024/11/19 18:29 | 文件夹 |    |  |           |
| 03_KEY                         | 2024/11/19 20:09 | 文件夹 |    |  |           |
| 04_ExtInterrupt                | 2024/11/19 21:15 | 文件夹 |    |  |           |
| 05_UART                        | 2024/11/20 19:41 | 文件夹 |    |  |           |
| 06_Timer                       | 2024/11/20 20:28 | 文件夹 |    |  |           |
| 07_PWM                         | 2024/11/21 15:59 | 文件夹 |    |  |           |
| 08_ADC                         | 2024/11/21 18:37 | 文件夹 |    |  |           |
| 09_DMA                         | 2024/11/21 19:48 | 文件夹 |    |  |           |
| 10_I2C                         | 2024/11/25 21:03 | 文件夹 |    |  |           |
| docs                           | 2024/11/14 15:05 | 文件夹 |    |  |           |
| examples                       | 2024/11/14 15:05 | 文件夹 |    |  |           |

# 1. 打开SYSCONFIG配置工具

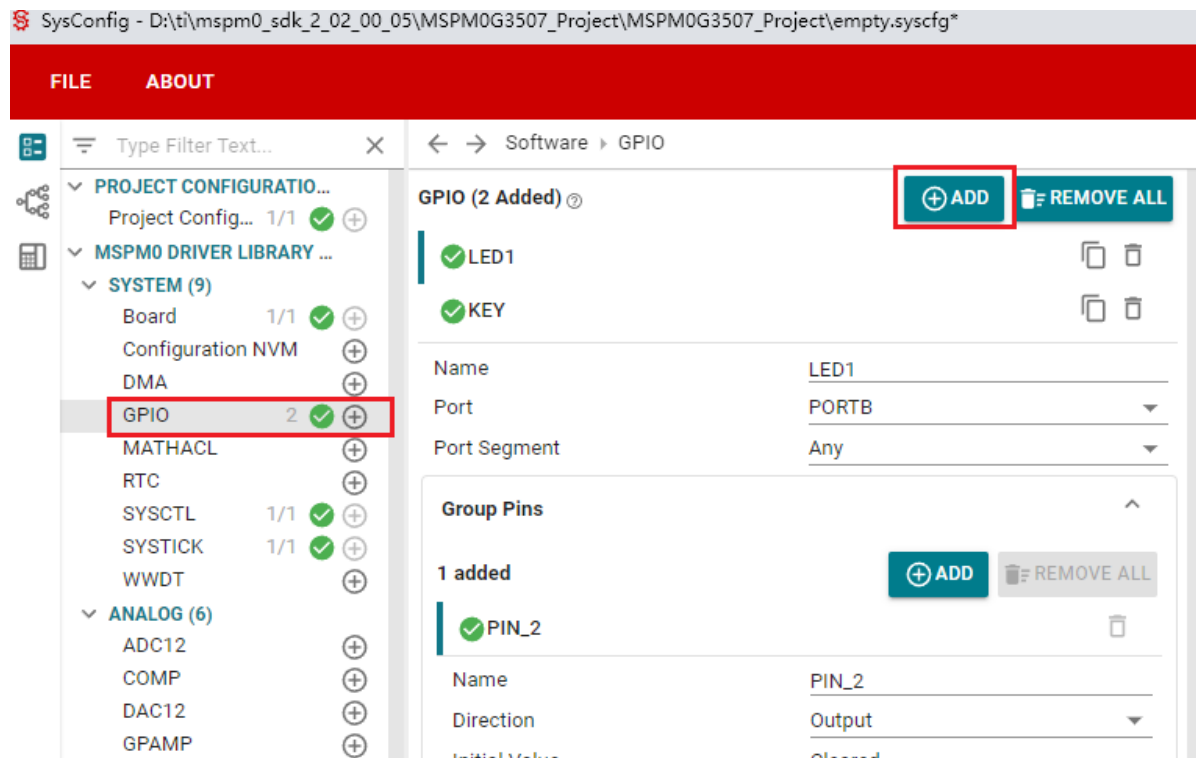
接着打开该工程。

| ti > mspm0_sdk_2_02_00_05 > MSPM0G3507_Project > MSPM0G3507_Project > keil |                  |                    |       |  | 在 |
|----------------------------------------------------------------------------|------------------|--------------------|-------|--|---|
| 名称                                                                         | 修改日期             | 类型                 | 大小    |  |   |
| Listings                                                                   | 2024/11/25 20:49 | 文件夹                |       |  |   |
| Objects                                                                    | 2024/11/25 20:49 | 文件夹                |       |  |   |
| empty_LP_MSPM0G3507_nortos_keil....                                        | 2024/11/25 20:50 | ADMINISTRATO...    | 90 KE |  |   |
| empty_LP_MSPM0G3507_nortos_keil....                                        | 2024/6/26 14:16  | UVOPTX 文件          | 21 KE |  |   |
| empty_LP_MSPM0G3507_nortos_keil...                                         | 2024/6/26 11:07  | vision5 Project    | 20 KE |  |   |
| mspm0g3507.sct                                                             | 2024/1/25 11:42  | Windows Script ... | 1 KE  |  |   |
| startup_mspm0g350x_uvision.s                                               | 2024/1/25 11:42  | Assembler Source   | 10 KE |  |   |

在Keil的主界面打开empty.syscfg文件，在empty.syscfg文件打开的情况下，再打开SYSCONFIG的GUI界面。

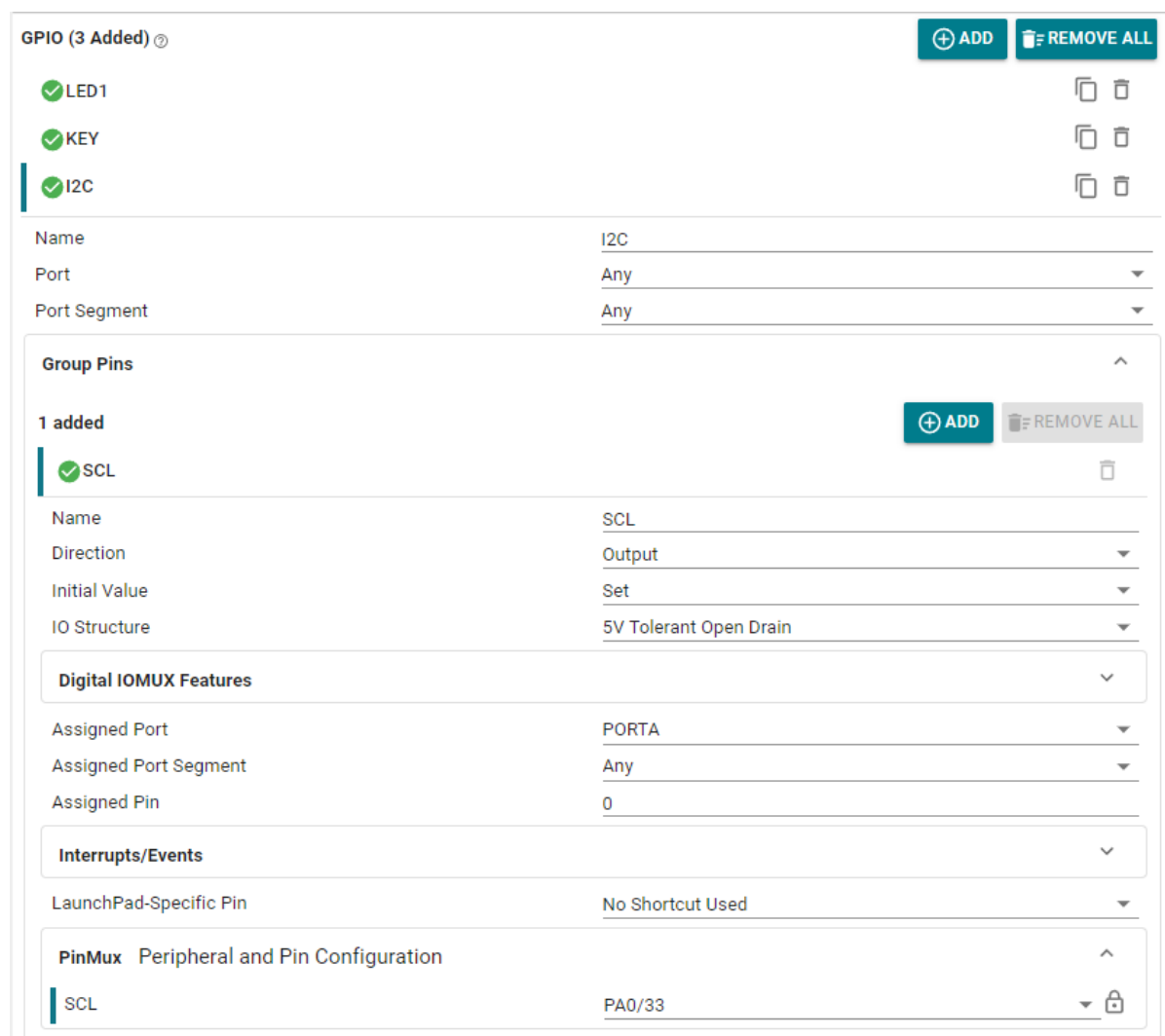


在左侧找到 GPIO 栏，点击进入，添加一组GPIO。



## 2. 引脚参数配置

SCL:



SDA:

GPIO (3 Added) ⓘ

+

ADD

🗑️

REMOVE ALL

✓LED1

✓KEY

✓I2C

📄

🗑️

📄

🗑️

📄

🗑️

Name

I2C

Port

Any

Port Segment

Any

Group Pins

^

2 added

+

ADD

🗑️

REMOVE ALL

✓SCL

✓SDA

🗑️

🗑️

Name

SDA

Direction

Output

Initial Value

Set

IO Structure ⓘ

5V Tolerant Open Drain

Digital IOMUX Features

▼

Assigned Port

PORTA

Assigned Port Segment

Any

Assigned Pin

1

Interrupts/Events

▼

LaunchPad-Specific Pin

No Shortcut Used

PinMux Peripheral and Pin Configuration

^

SDA

Any(PA1/34)

🔒

Other Dependencies

▼

### 3. 串口配置

本课程是使用I2C协议通过串口将MPU6050的数据发送给上位机（电脑）显示。所以我们还需要配置串口。

未显示的部分为默认选项。

PROJECT CONFIGURATION...  
Project Config... 1/1

MSPM0 DRIVER LIBRARY ...

SYSTEM (9)  
Board 1/1   
Configuration NVM   
DMA   
GPIO 3   
MATHACL   
RTC   
SYSCTL 1/1   
SYSTICK 1/1   
WWDT   
ANALOG (6)  
ADC12   
COMP   
DAC12   
GPAMP   
OPA   
VREF   
COMMUNICATIONS (6)  
I2C   
I2C - SMBUS   
MCAN   
SPI   

UART 1/4

UART - LIN

UART (1 of 4 Added) ②

ADD

REMOVE ALL

UART\_0

Name

UART\_0

Selected Peripheral

UART0

Quick Profiles

UART Profiles

Custom

Basic Configuration

UART Initialization Configuration

Clock Source

MFCLK

Clock Divider

Divide by 1

Calculated Clock Source

4.00 MHz

Target Baud Rate

9600

Calculated Baud Rate

9598.08

Calculated Error (%)

0.02

Word Length

8 bits

Parity

None

Stop Bits

One

HW Flow Control

Disable HW flow control

Advanced Configuration

UART Mode

Normal UART Mode

Communication Direction

TX and RX

Oversampling

16x

Enable FIFOs

☐

Analog Glitch Filter

Disabled

Digital Glitch Filter

0

Calculated Digital Glitch Filter

0.00 s

RX Timeout Interrupt Counts

0

Calculated RX Timeout Interrupt

0.00 s

Enable Internal Loopback

☐

Enable Majority Voting

☐

Enable MSB First

☐

Retention Configuration

Low-Power Register Retention

Registers retained

Disable Retention APIs

☐

Extend Configuration

Interrupt Configuration

Enable Interrupts

Receive

Interrupt Priority

Default

|                                         |         |     |
|-----------------------------------------|---------|-----|
| PinMux Peripheral and Pin Configuration |         | ^   |
| UART Peripheral                         | UART0   | ▼ 🔒 |
| RX Pin                                  | PA11/57 | ▼ 🔒 |
| TX Pin                                  | PA10/56 | ▼ 🔒 |

## 4. I2C协议使用

我们在工程模板中新建两个文件，分别是 bsp\_mpu6050.c 和 bsp\_mpu6050.h。将它们保存在工程的 BSP 文件夹下。

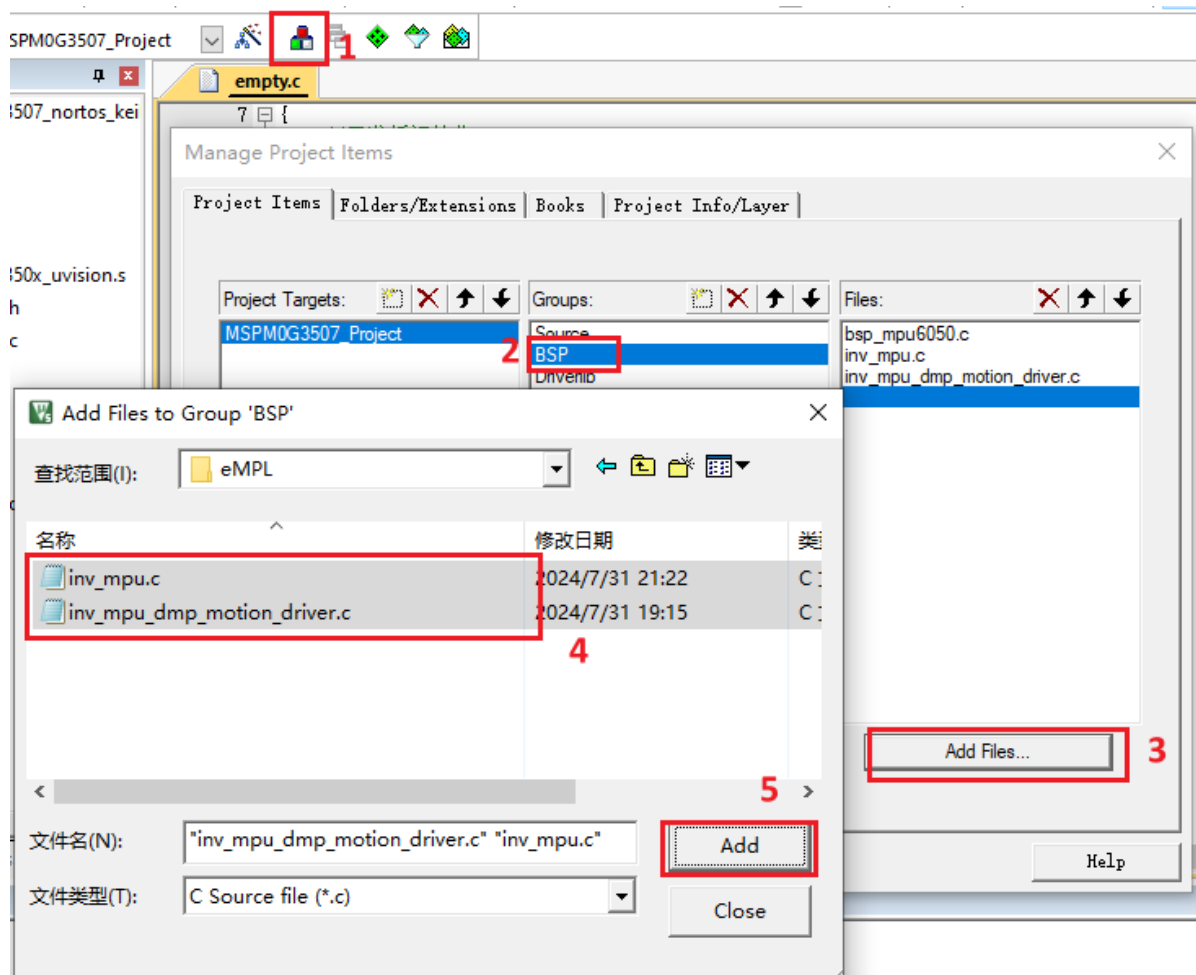
|                                                      |                  |              |       |  |     |             |
|------------------------------------------------------|------------------|--------------|-------|--|-----|-------------|
| 加卷 (D:) > ti > mspm0_sdk_2_02_00_05 > 10_I2C > BSP > |                  |              |       |  | ▼ 🔒 | 在 BSP ... 🔍 |
| 名称                                                   | 修改日期             | 类型           | 大小    |  |     |             |
| bsp_mpu6050.h                                        | 2024/11/26 10:38 | C Header 源文件 | 4 KB  |  |     |             |
| bsp_mpu6050.c                                        | 2024/11/26 10:16 | C 文件         | 13 KB |  |     |             |

包括以下这些MPU6050陀螺仪和加速度计设计的驱动程序，放入eMPL文件夹中

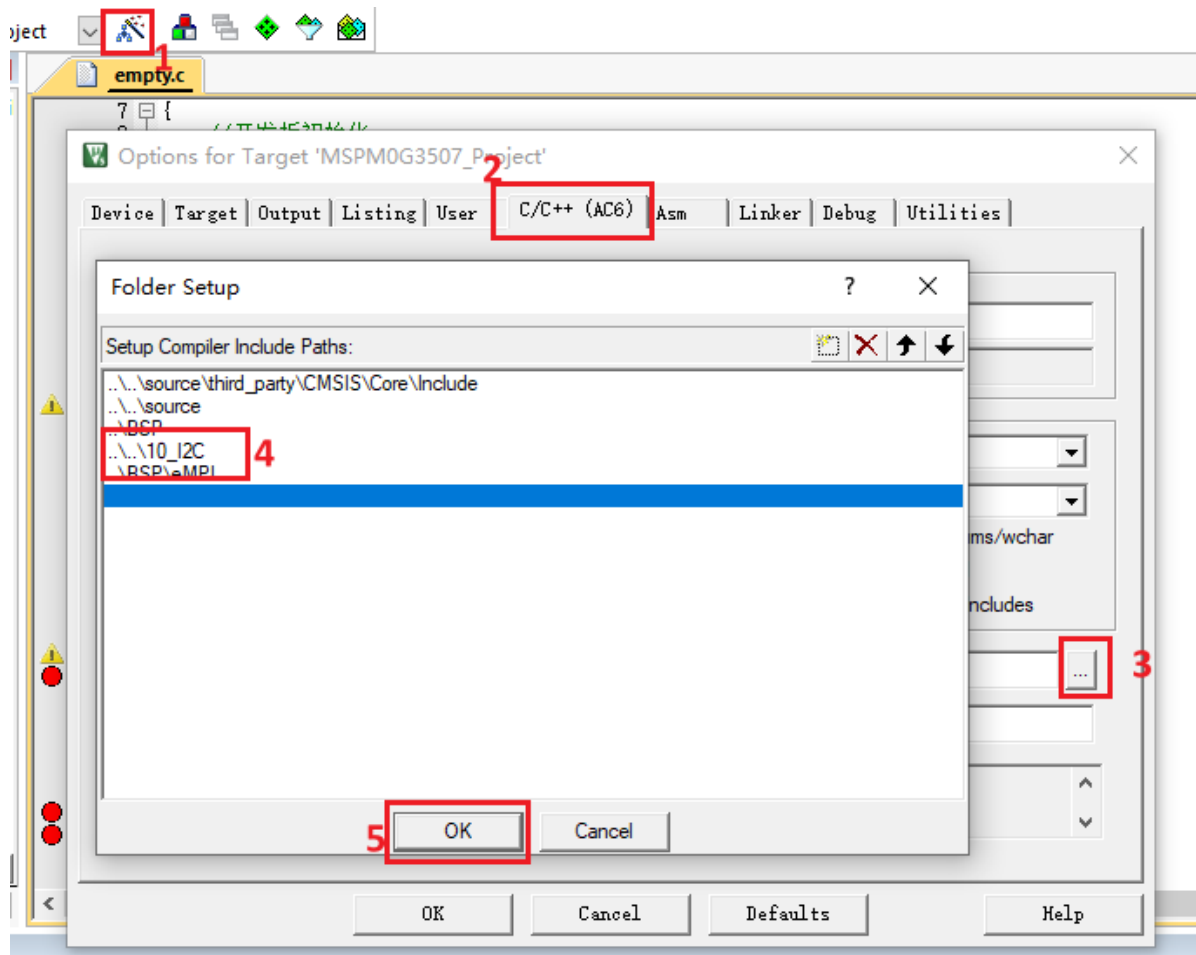
|                                                           |                 |              |       |  |     |           |
|-----------------------------------------------------------|-----------------|--------------|-------|--|-----|-----------|
| 加卷 (D:) > ti > mspm0_sdk_2_02_00_05 > 10_I2C > BSP > eMPL |                 |              |       |  | ▼ 🔒 | 在 eMPL .. |
| 名称                                                        | 修改日期            | 类型           | 大小    |  |     |           |
| dmpKey.h                                                  | 2024/7/31 19:15 | C Header 源文件 | 19 KB |  |     |           |
| dmpmap.h                                                  | 2024/7/31 19:15 | C Header 源文件 | 7 KB  |  |     |           |
| inv_mpu.c                                                 | 2024/7/31 21:22 | C 文件         | 87 KB |  |     |           |
| inv_mpu.h                                                 | 2024/7/31 19:15 | C Header 源文件 | 5 KB  |  |     |           |
| inv_mpu_dmp_motion_driver.c                               | 2024/7/31 19:15 | C 文件         | 57 KB |  |     |           |
| inv_mpu_dmp_motion_driver.h                               | 2024/7/31 19:15 | C Header 源文件 | 4 KB  |  |     |           |

打开项目管理，点击BSP，在BSP中添加我们之前新建和复制过来的文件。





更新头文件路径。



## 5. 写入程序

在使用软件实现I2C通信时，需要选择合适的引脚来作为数据线（SDA）和时钟线（SCL）。通常情况下，可以选择任何可编程的通用输入输出（GPIO）引脚作为软件I2C的引脚。对于软件I2C，需要至少两个引脚用于数据线（SDA）和时钟线（SCL），并确保这些引脚能够满足I2C通信协议的时序要求。以下是一般的引脚说明：

1. 数据线（SDA）：用于传输数据的引脚。在软件I2C中，需要将该引脚设置为输出模式（用于主设备发送数据）与输入模式（用于主设备接收数据）。在通信期间，需要通过控制数据线的电平变化来实现数据的传输。
2. 时钟线（SCL）：用于控制数据传输的时钟信号的引脚。在软件I2C中，需要将该引脚设置为输出模式，通过控制时钟线的电平变化来生成时钟脉冲，以控制数据线的传输。需要注意的是，选取合适的引脚时要考虑以下几个方面：
  - 支持输入/输出配置：引脚需要支持在软件中配置为输入或输出模式，并能够通过程序动态地进行切换。
  - 硬件限制和冲突：确保选取的引脚没有被分配给其他硬件功能或外设，以避免冲突。
  - 电气特性：引脚的电气特性应满足I2C总线的标准要求，例如正确的电平和驱动能力。

需要注意的是，软件I2C的实现需要更多的程序代码和计算，相对于硬件I2C，软件I2C在处理器效能和时序控制方面更加敏感。因此，在选择引脚时，还需要考虑处理器的性能和可编程性能。为了保证代码的可维护性和可移植性，这里将相关的功能进行的宏定义。

SDA引脚和SCL引脚的宏定义如下：

bsp\_mpu6050.h

```
#ifndef _BSP_MPU6050_H_
#define _BSP_MPU6050_H_

#include "board.h"

//设置SDA输出模式 Set SDA output mode
#define SDA_OUT() { \
    DL_GPIO_initDigitalOutput(I2C_SDA_IOMUX); \
    DL_GPIO_setPins(I2C_PORT, I2C_SDA_PIN); \
    DL_GPIO_enableOutput(I2C_PORT, I2C_SDA_PIN); \
}

//设置SDA输入模式 Set SDA input mode
#define SDA_IN() { DL_GPIO_initDigitalInput(I2C_SDA_IOMUX); }

//获取SDA引脚的电平变化 Get the level change of the SDA pin
#define SDA_GET() ( ( ( DL_GPIO_readPins(I2C_PORT, I2C_SDA_PIN) & I2C_SDA_PIN ) \
> 0 ) ? 1 : 0 )

//SDA与SCL输出 SDA and SCL output
#define SDA(x) ( (x) ? (DL_GPIO_setPins(I2C_PORT, I2C_SDA_PIN)) : \
(DL_GPIO_clearPins(I2C_PORT, I2C_SDA_PIN)) )
#define SCL(x) ( (x) ? (DL_GPIO_setPins(I2C_PORT, I2C_SCL_PIN)) : \
(DL_GPIO_clearPins(I2C_PORT, I2C_SCL_PIN)) )

//MPU6050的AD0是IIC地址引脚，接地则IIC地址为0x68,接VCC则IIC地址为0x69
//AD0 of MPU6050 is the IIC address pin. If it is grounded, the IIC address is \
0x68. If it is connected to VCC, the IIC address is 0x69.
```

```

#define MPU6050_RA_SMPLRT_DIV      0x19      //陀螺仪采样率 地址      Gyro
sampling rate address
#define MPU6050_RA_CONFIG          0x1A      //设置数字低通滤波器 地址  Set
digital low-pass filter address
#define MPU6050_RA_GYRO_CONFIG     0x1B      //陀螺仪配置寄存器      Gyro
configuration register
#define MPU6050_RA_ACCEL_CONFIG    0x1C      //加速度传感器配置寄存器
Acceleration sensor configuration register
#define MPU_INT_EN_REG              0x38      //中断使能寄存器      Interrupt
enable register
#define MPU_USER_CTRL_REG          0x6A      //用户控制寄存器      User control
register
#define MPU_FIFO_EN_REG            0x23      //FIFO使能寄存器      FIFO enable
register
#define MPU_PWR_MGMT2_REG          0x6C      //电源管理寄存器2      Power
management register 2
#define MPU_GYRO_CFG_REG           0x1B      //陀螺仪配置寄存器      Gyroscope
configuration register
#define MPU_ACCEL_CFG_REG          0x1C      //加速度计配置寄存器
Accelerometer configuration register
#define MPU_CFG_REG                0x1A      //配置寄存器      Configuration
register
#define MPU_SAMPLE_RATE_REG        0x19      //采样频率分频器      Sampling
frequency divider
#define MPU_INTBP_CFG_REG          0x37      //中断/旁路设置寄存器
Interrupt/bypass setting register

#define MPU6050_RA_PWR_MGMT_1      0x6B
#define MPU6050_RA_PWR_MGMT_2      0x6C

#define MPU6050_WHO_AM_I           0x75
#define MPU6050_SMPLRT_DIV         0          //8000Hz
#define MPU6050_DLPF_CFG           0
#define MPU6050_GYRO_OUT           0x43      //MPU6050陀螺仪数据寄存器地址
MPU6050 gyroscope data register address
#define MPU6050_ACC_OUT            0x3B      //MPU6050加速度数据寄存器地址
MPU6050 acceleration data register address

#define MPU6050_RA_TEMP_OUT_H       0x41      //温度高位 High temperature
#define MPU6050_RA_TEMP_OUT_L       0x42      //温度低位 Low temperature

#define MPU_ACCEL_XOUTH_REG         0x3B      //加速度值,X轴高8位寄存器
Acceleration value, x-axis high 8-bit register
#define MPU_ACCEL_XOUTL_REG         0x3C      //加速度值,X轴低8位寄存器
Acceleration value, x-axis low 8-bit register
#define MPU_ACCEL_YOUTH_REG         0x3D      //加速度值,Y轴高8位寄存器
Acceleration value, y-axis high 8-bit register
#define MPU_ACCEL_YOUTL_REG         0x3E      //加速度值,Y轴低8位寄存器
Acceleration value, y-axis low 8-bit register
#define MPU_ACCEL_ZOUTH_REG         0x3F      //加速度值,Z轴高8位寄存器
Acceleration value, z-axis high 8-bit register
#define MPU_ACCEL_ZOUTL_REG         0x40      //加速度值,Z轴低8位寄存器
Acceleration value, z-axis low 8-bit register

#define MPU_TEMP_OUTH_REG           0x41      //温度值高八位寄存器 Temperature
value high eight bits register
#define MPU_TEMP_OUTL_REG           0x42      //温度值低8位寄存器 Temperature
value lower 8 bits register

```

```

#define MPU_GYRO_XOUTH_REG      0x43      //陀螺仪值,X轴高8位寄存器 Gyroscope
value, X-axis high 8-bit register
#define MPU_GYRO_XOUTL_REG      0x44      //陀螺仪值,X轴低8位寄存器 Gyroscope
value, X-axis low 8-bit register
#define MPU_GYRO_YOUTH_REG      0x45      //陀螺仪值,Y轴高8位寄存器 Gyroscope
value, Y-axis high 8-bit register
#define MPU_GYRO_YOUTL_REG      0x46      //陀螺仪值,Y轴低8位寄存器 Gyroscope
value, Y-axis low 8-bit register
#define MPU_GYRO_ZOUTH_REG      0x47      //陀螺仪值,Z轴高8位寄存器 Gyroscope
value, Z-axis high 8-bit register
#define MPU_GYRO_ZOUTL_REG      0x48      //陀螺仪值,Z轴低8位寄存器 Gyroscope
value, Z-axis low 8-bit register

char MPU6050_WriteReg(uint8_t addr,uint8_t regaddr,uint8_t num,uint8_t
*regdata);
char MPU6050_ReadData(uint8_t addr, uint8_t regaddr,uint8_t num,uint8_t* Read);

char MPU6050_Init(void);
void MPU6050ReadGyro(short *gyroData);
void MPU6050ReadAcc(short *accData);
float MPU6050_GetTemp(void);
uint8_t MPU6050ReadID(void);
#endif

```

接下来就是配置I2C的时序部分，

bsp\_mpu6050.c（这里只截取部分，详细内容请查看工程源码）

```

#include "bsp_mpu6050.h"
#include "stdio.h"

/*****
* 函数名称: IIC_Start
* 函数说明: IIC起始时序
* 函数形参: 无
* 函数返回: 无
* 作者: LC
* 备注: 无
* Function name: IIC_Start
* Function description: IIC start timing
* Function parameter: None
* Function return: None
* Author: LC
* Remarks: None
*****/
void IIC_Start(void)
{
    SDA_OUT();
    SCL(1);
    SDA(0);

    SDA(1);
    delay_us(5);
    SDA(0);
}

```

```

        delay_us(5);

        SCL(0);
    }
    /*****
    * 函数名称: IIC_Stop
    * 函数说明: IIC停止信号
    * 函数形参: 无
    * 函数返回: 无
    * 作者: LC
    * 备注: 无
    * Function name: IIC_Stop
    * Function description: IIC stop signal
    * Function parameters: None
    * Function return: None
    * Author: LC
    * Notes: None
    *****/
void IIC_Stop(void)
{
    SDA_OUT();
    SCL(0);
    SDA(0);

    SCL(1);
    delay_us(5);
    SDA(1);
    delay_us(5);

}
...

```

之后在empty.c文件中，写入以下代码

```

#include "board.h"
#include <stdio.h>
#include "bsp_mpu6050.h"
#include "inv_mpu.h"

int main(void)
{
    //开发板初始化 Development board initialization
    board_init();

    MPU6050_Init();

    uint8_t ret = 1;

    float pitch=0,roll=0,yaw=0;    //欧拉角 Euler Angles

    printf("start\r\n");

    //DMP初始化 DMP Initialization
    while( mpu_dmp_init() )
    {
        printf("dmp error\r\n");
        delay_ms(200);
    }
}

```

```

}

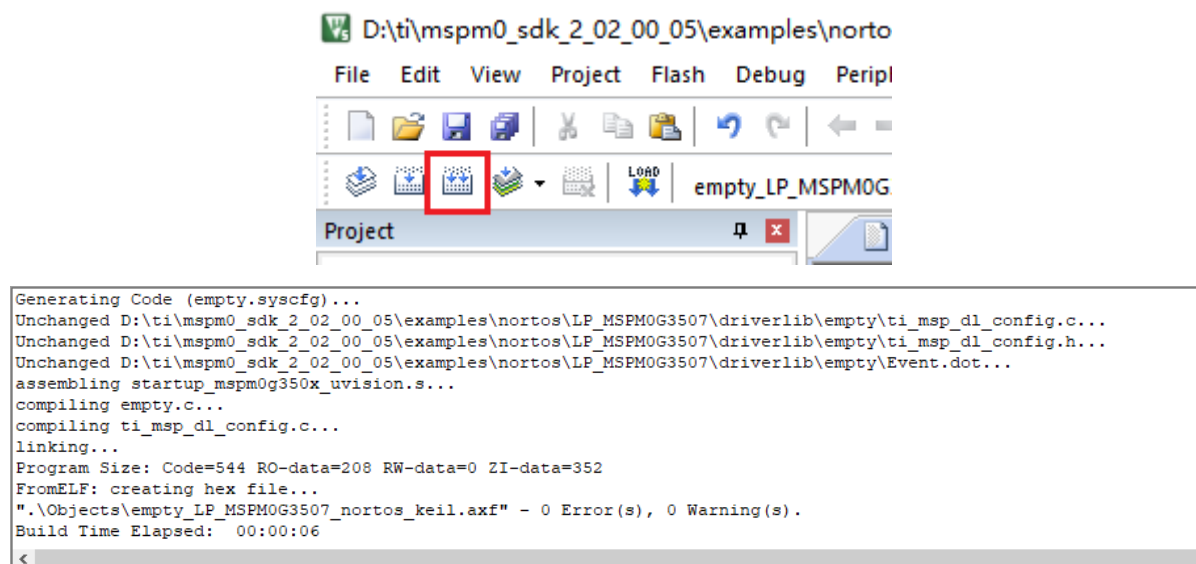
printf("Initialization Data Succeed \r\n");

while(1)
{
    //获取欧拉角 Get Euler angles
    if( mpu_dmp_get_data(&pitch,&roll,&yaw) == 0 )
    {
        printf("\r\npitch =%d\r\n", (int)pitch);
        printf("\r\nroll =%d\r\n", (int)roll);
        printf("\r\nyaw =%d\r\n", (int)yaw);
    }
    delay_ms(200); //根据设置的采样率，不可设置延时过大 According to the set
    sampling rate, the delay cannot be set too large
}
}

```

## 6. 编译

点击 Rebuild 图标。出现以下提示表示编译完成，并且没有错误。



## 四、程序分析

- inv\_mpu.c

```

//mpu6050,dmp初始化
//返回值:0,正常
//    其他,失败
//mpu6050, dmp initialization
//Return value: 0, normal
//    other, failed
u8 mpu_dmp_init(void)
{
    u8 res=0;

    res = mpu_init();
    //    printf("res = %d\r\n",res);
    if(res==0) //初始化MPU6050 Initialize MPU6050

```

```

{
    res=mpu_set_sensors(INV_XYZ_GYRO|INV_XYZ_ACCEL); //设置所需要的传感器 Set up
the required sensors
    if(res)return 1;
    res=mpu_configure_fifo(INV_XYZ_GYRO | INV_XYZ_ACCEL); //设置FIFO Setting
up FIFO
    if(res)return 2;
    res=mpu_set_sample_rate(DEFAULT_MPU_HZ); //设置采样率 Setting the
Sample Rate
    if(res)return 3;
    res=dmp_load_motion_driver_firmware(); //加载dmp固件 Load dmp
firmware
    if(res)return 4;

    res=dmp_set_orientation(inv_orientation_matrix_to_scalar(gyro_orientation)); //设
置陀螺仪方向 Set gyroscope orientation
    if(res)return 5;
    res=dmp_enable_feature(DMP_FEATURE_6X_LP_QUAT|DMP_FEATURE_TAP| //设置dmp
功能 Setting the dmp function

    DMP_FEATURE_ANDROID_ORIENT|DMP_FEATURE_SEND_RAW_ACCEL|DMP_FEATURE_SEND_CAL_GYRO
    |
        DMP_FEATURE_GYRO_CAL);
    if(res)return 6;
    res=dmp_set_fifo_rate(DEFAULT_MPU_HZ); //设置DMP输出速率(最大不超过200Hz)
Set the DMP output rate (maximum 200Hz)
    if(res)return 7;
    // res=run_self_test(); //自检 Self-Test
    // if(res)return 8;
    res=mpu_set_dmp_state(1); //使能DMP Enabling DMP
    if(res)return 9;
}

return 0;
}

```

这段代码实现了对 MPU6050 的初始化，通过 DMP 提供高效的运动追踪和姿态解算能力。。调用 `mpu_init` 初始化硬件模块；启用陀螺仪和加速度计并配置其采样率和 FIFO 缓存；加载 DMP 固件以支持高级功能（如姿态解算和手势检测）；设置设备方向矩阵和 DMP 的功能特性（如四元数计算、数据校准等）；最后启用 DMP 输出，使设备可以以设定的频率提供稳定的姿态和传感器数据。

```

//得到dmp处理后的数据(注意,本函数需要比较多堆栈,局部变量有点多)
//pitch:俯仰角 精度:0.1° 范围:-90.0° <---> +90.0°
//roll:横滚角 精度:0.1° 范围:-180.0°<---> +180.0°
//yaw:航向角 精度:0.1° 范围:-180.0°<---> +180.0°
//返回值:0,正常
// 其他,失败
//Get the data after dmp processing (note that this function requires a lot of
stacks and a lot of local variables)
//pitch: pitch angle accuracy: 0.1° range: -90.0° <---> +90.0°
//roll: roll angle accuracy: 0.1° range: -180.0° <---> +180.0°
//yaw: heading angle accuracy: 0.1° range: -180.0° <---> +180.0°
//Return value: 0, normal
// others, failed
u8 mpu_dmp_get_data(float *pitch,float *roll,float *yaw)
{
    float q0=1.0f,q1=0.0f,q2=0.0f,q3=0.0f;

```

```

    unsigned long sensor_timestamp;
    short gyro[3], accel[3], sensors;
    unsigned char more;
    long quat[4];
    if(dmp_read_fifo(gyro, accel, quat, &sensor_timestamp,
&sensors,&more))return 1;
    /* Gyro and accel data are written to the FIFO by the DMP in chip frame and
hardware units.
    * This behavior is convenient because it keeps the gyro and accel outputs
of dmp_read_fifo and mpu_read_fifo consistent.
    */
    /*if (sensors & INV_XYZ_GYRO )
    send_packet(PACKET_TYPE_GYRO, gyro);
    if (sensors & INV_XYZ_ACCEL)
    send_packet(PACKET_TYPE_ACCEL, accel); */
    /* Unlike gyro and accel, quaternions are written to the FIFO in the body
frame, q30.
    * The orientation is set by the scalar passed to dmp_set_orientation during
initialization.
    */
    if(sensors&INV_WXYZ_QUAT)
    {
        q0 = quat[0] / q30; //q30格式转换为浮点数 Convert q30 format to floating
point number
        q1 = quat[1] / q30;
        q2 = quat[2] / q30;
        q3 = quat[3] / q30;
        //计算得到俯仰角/横滚角/航向角
        // Calculate the pitch angle/roll angle/heading angle
        *pitch = asin(-2 * q1 * q3 + 2 * q0* q2)* 57.3; // pitch
        *roll = atan2(2 * q2 * q3 + 2 * q0 * q1, -2 * q1 * q1 - 2 * q2* q2 +
1)* 57.3; // roll
        *yaw = atan2(2*(q1*q2 + q0*q3),q0*q0+q1*q1-q2*q2-q3*q3) * 57.3;
    //yaw
    }else return 2;
    return 0;
}

```

该函数从 MPU6050 的 FIFO 中读取 DMP 处理后的传感器数据（包括陀螺仪、加速度计和四元数），通过解析四元数计算出俯仰角（pitch）、横滚角（roll）和航向角（yaw），并将结果返回。

- empty.c

```

int main(void)
{
    //开发板初始化 Development board initialization
    board_init();

    MPU6050_Init();

    uint8_t ret = 1;

    float pitch=0,roll=0,yaw=0;    //欧拉角 Euler Angles

    printf("start\r\n");

    //DMP初始化 DMP Initialization

```



```

while( mpu_dmp_init() )
{
    printf("dmp error\r\n");
    delay_ms(200);
}

printf("Initialization Data Succeed \r\n");

while(1)
{
    //获取欧拉角 Get Euler angles
    if( mpu_dmp_get_data(&pitch,&roll,&yaw) == 0 )
    {
        printf("\r\npitch =%d\r\n", (int)pitch);
        printf("\r\nroll =%d\r\n", (int)roll);
        printf("\r\nyaw =%d\r\n", (int)yaw);
    }
    delay_ms(200); //根据设置的采样率，不可设置延时过大 According to the set
    sampling rate, the delay cannot be set too large
}
}

```

该程序实现了 MPU6050 的实时姿态角测量功能。通过初始化开发板和 MPU6050，并加载 DMP 模块，计算设备的俯仰角（pitch）、横滚角（roll）和航向角（yaw）。程序先进行 DMP 初始化，失败时重试，成功后进入主循环，不断读取姿态角数据并通过串口打印输出。

## 五、实验现象

程序下载完成之后，对串口助手进行如下图配置之后，打开串口，可以读取到 MPU6050 模块的实时数据。



